

Course: EPPS 6354 - Information Management

Instructor: Dr. Karl Ho

Student: Richa Gandhi

Date: May 2026

Introduction

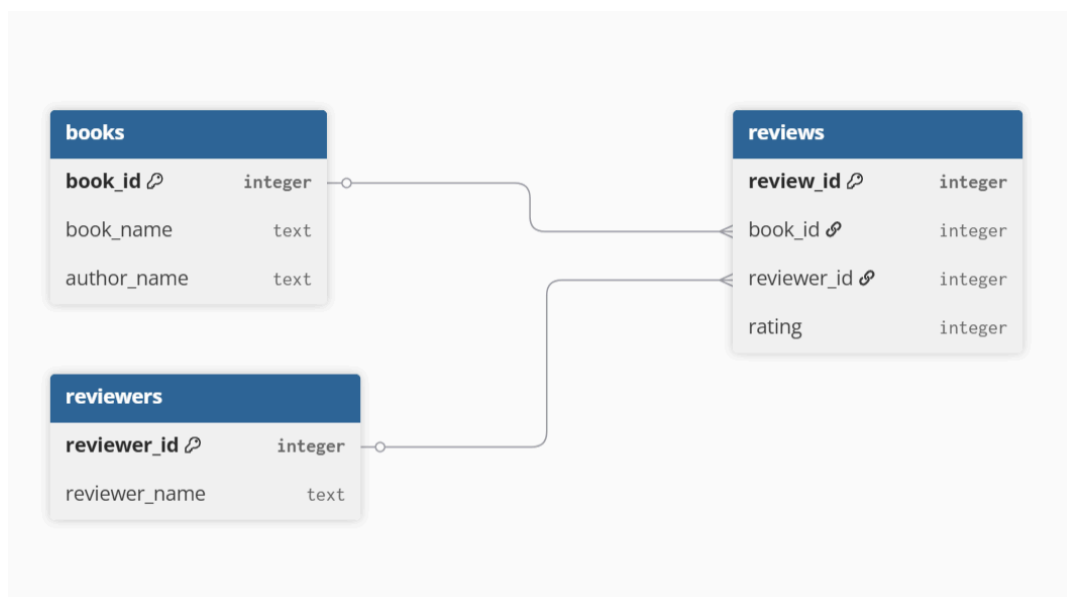
When deciding on my topic of choice for my Database project, I realized that although we have millions of books available to us, it's still quite difficult to find a book that actually resonates with our personal taste. We are often faced with the problem of the “paradox of choice”. This is a problem that I run into often as an avid reader, and I end up choosing books based on author, genre, or the hottest/trending ones occurring at the time. Additionally, recommendation tools that are available only rely on global averages, which fail to capture the essence of an individual's preferences. Originally, the goal of this project was to solve this problem by building a search engine discovery tool and intended to serve as a broad social platform where you can type in a vibe/feeling you would want in a book, and that recommendation would pop up. This involved using a complex mix of Vector databases and NoSQL called the “R-Reader Engine” to capture community memory from group notes and book clubs. I wanted to solve this problem by using fragmented data and being a predictor for genre burnout through emotional tone scoring and genre burnout.

However, as I started working in the development phase of this project, it came to my attention that I needed to make changes in the direction it was headed. In the timeframe between the proposal and when this project was due, the social-heavy platform was too broad and risked making the data hard to manage and making the data messy. Instead of a search engine, I decided to pivot towards building a relational SQL foundation that would provide high-performance recommendations from this engine. By making this pivot, I was able to turn my focus towards data accuracy, mathematical vectors used for cosine similarity, and its ability to be a predictive analyzer rather than a search engine tool. This felt best for the objectives of this project and course, as we have learned a lot about structured relationships between entities in relational

database schema, while serving to be professionally robust and technically attainable. The final product for this project is now called the Vector Verse.

Database Design and Cloud-Based Integration

A major technical goal that I had for this project was to ensure that the application that was built has the ability to remember users and their ratings across different devices and sessions. In order to make this happen, I built a cloud-hosted infrastructure and moved away from using local files. This was done by building a relational PostgreSQL database hosted on Neon, where I cleaned and gathered large datasets from Kaggle and my own manual search from sites such as Amazon, Goodreads, and Reddit for the library. With large volumes of data, it is easy for it to quickly become unorganized; therefore, I split it into three tables: Reviewers, Reviews, and Books. The Reviewers table acts as a directory for user names, the reviews table acts as the connection between users to the books they have read, and lastly, the books table serves as a library catalog. Efficient querying and data integrity were achieved through this structure.



```

2 v CREATE TABLE books (
3     book_id SERIAL PRIMARY KEY,
4     title VARCHAR(255) NOT NULL,
5     author VARCHAR(100),
6     genre VARCHAR(50),
7     description TEXT,
8     published_year INTEGER

2 v CREATE TABLE reviewers (
3     reviewer_id SERIAL PRIMARY KEY,
4     reviewer_name TEXT

15 v CREATE TABLE reviews (
16     review_id SERIAL PRIMARY KEY,
17     book_id INT REFERENCES books(book_id),
18     reviewer_id INT REFERENCES reviewers(reviewer_id),
19     rating INT CHECK (rating >= 1 AND rating <= 5)

```

```

1 v SELECT b.book_name, COUNT(r.review_id) as total_reviews, AVG(r.rating) as avg_score
2 FROM books b
3 JOIN reviews r ON b.book_id = r.book_id
4 GROUP BY b.book_name
5 ORDER BY avg_score DESC
6 LIMIT 10;

```

```

1 v SELECT
2     b.book_name,
3     b.author_name,
4     r.rating
5 FROM reviews r
6 JOIN books b ON r.book_id = b.book_id
7 JOIN reviewers rev ON r.reviewer_id = rev.reviewer_id
8 WHERE rev.reviewer_name = 'Richa Gandhi';

```

```

1 v SELECT r.review_id, v.reviewer_name, b.book_name, r.rating
2 FROM reviews r
3 JOIN reviewers v ON r.reviewer_id = v.reviewer_id
4 JOIN books b ON r.book_id = b.book_id
5 WHERE r.reviewer_id > 813;

```

A technique that I used to make sure the system works correctly is called Atomic transactions. It was used through the `dbWithTransaction` function and ensures that the system treats two actions

as a single move when a user adds a new book and rating at the same time. If the system crashes or an error happens halfway through, it cancels the whole move rather than saving half of it. As we have learned in class, this satisfies transactions of ACID properties: atomicity (process must happen in its entirety or not at all), consistency, isolation, and durability. With this transaction installed, data that has a rating floating around with no book attached to it wouldn't occur. The key to professional database management is that the library stays reliable and clean when users access the app and add their own data.

Book Recommender

All OK Ask AI Upgrade R

Tables

production

neondb

Database studio

public

Search...

books

reviewers

reviews

book_id serial	book_name text	author_name text	reviews
1	A Court of Mist and Fu...	Sarah J. Maas	reviews
2	A Court of Thorns and ...	Sarah J. Maas	reviews
3	A Court of Wings and R...	Sarah J. Maas	reviews
4	A Little Life	Hanya Yanagihara	reviews
5	All the Light We Canno...	Anthony Doerr	reviews
6	Atomic Habits	James Clear	reviews
7	Balloons over Broadway	Melissa Sweet	reviews
8	Brown Bear, Brown Bear...	Bill Martin Jr. & Eric...	reviews
9	Chicka Chicka Boom Boom	Bill Martin Jr. & John...	reviews
10	Demon Copperhead	Barbara Kingsolver	reviews
11	Dirty Thirty (Stephani...	Janet Evanovich	reviews
12	Don't Let the Pigeon D...	Mo Willems	reviews
13	Elon Musk	Walter Isaacson	reviews
14	First Little Readers P...	Deborah Schecter	reviews
15	Five Silly Turkeys	Salina Yoon	reviews
16	Flash Cards: Sight Wor...	Scholastic Teacher Res...	reviews
17	Forever Strong	Dr. Gabrielle Lyon	reviews
18	Fourth Wing	Rebecca Yarros	reviews
19	Friends, Lovers, and t...	Matthew Perry	reviews
20	From Crook to Cook	Snoop Dogg	reviews
21	Goodnight Moon	Margaret Wise Brown	reviews
22	Goodnight, Goodnight C...	Sherri Duskey Rinker	reviews
23	Harry Potter and the P...	J.K. Rowling	reviews

50 rows • 187ms

Book Recommender

production

neondb Database studio

public

books reviewers reviews

50 rows • 184ms

reviewer_id	reviewer_name	reviews
1	*AT	reviews
2	A H Kobayashi	reviews
3	a reader	reviews
4	A Reviewer	reviews
5	A. K. P.	reviews
6	A. Slater	reviews
7	A.B.318	reviews
8	A.S.	reviews
9	a_bennett	reviews
10	AB1213	reviews
11	Abby Dye	reviews
12	Abigail	reviews
13	acpatrick	reviews
14	Adam	reviews
15	Adam Manning	reviews
16	Adam Nelson	reviews
17	Adam the book reader	reviews
18	Adrea Crosby	reviews
19	Agnes Shapiro	reviews
20	Aidan Longueville	reviews
21	aiden	reviews
22	Aimee	reviews

Book Recommender

production

neondb Database studio

public

books reviewers reviews

50 rows • 176ms

review_id	book_id	reviewer_id	rating	book	reviewer
1	26	1	5	book	reviewer
2	39	1	3	book	reviewer
3	14	1	5	book	reviewer
4	60	1	2	book	reviewer
5	42	1	2	book	reviewer
6	89	1	2	book	reviewer
7	52	1	4	book	reviewer
8	85	1	3	book	reviewer
9	21	1	3	book	reviewer
10	44	1	4	book	reviewer
11	6	2	4	book	reviewer
12	36	2	4	book	reviewer
13	46	2	1	book	reviewer
14	3	2	1	book	reviewer
15	30	2	2	book	reviewer
16	14	2	1	book	reviewer
17	7	2	5	book	reviewer
18	22	2	4	book	reviewer
19	60	2	1	book	reviewer
20	15	2	5	book	reviewer
21	82	3	4	book	reviewer
22	32	3	1	book	reviewer

Application

In the development of using R and Shiny, I encountered a technical obstacle. When extracting book review data from sources such as Goodreads, Amazon, and Reddit, I used as much authentic data from what I had found and incorporated it into the engine to support real-world applicability, but faced a limitation in that a majority of reviewers had only contributed to one or two ratings. In these beginning stages of analysis, which were revealed in

the dataset, there was a gap in order for the recommendation system to work because it relies on behavioral similarities between individuals, and not having a big enough ratio between reviewers and ratings created a lack of data density. There is no mathematical bridge if a user has only rated a single book or any history that can connect their book preferences to the rest of the community, making it much harder for the Cosine Similarity algorithm to work. With this initial issue, the Cosine Similarity that I wanted to implement could not draw meaningful conclusions.

To overcome this technical obstacle, I developed a pseudo-data pipeline to demonstrate a more established long-term user base. Throughout this process, I kept existing book titles and reviewers and added additional randomized ratings to fill in the gaps in order to increase the robustness of the dataset. This was a vital step to implement in order to allow a mathematical model to look past basic metadata and for it to actually analyze behavioral patterns instead. It provides a healthier history of reviews and simulates a more mature system. For a recommendation system to be useful, there needs to be a sufficient data set for it to be useful for the users. By adding a more robust data structure, the system can now provide personalized recommendations based on patterns of preference that are mathematically backed up.

The front-end architecture of the application is built on the Shiny framework on the principles of reactive programming. Since implementing Shiny is a part of the project requirement, a specific challenge arose because the application is connected to the Neon cloud database. The issue with Shiny is that it is not designed to automatically detect updates or changes made by other users in a remote environment. Because of this problem, I implemented a custom refresh trigger logic. When a user registers a new account or submits a new rating, it forces the application to requery the database and ensures that information that pops up on the interface is an accurate reflection of what the current cloud state is, regardless of when the data was last

pulled. The user interface utilizes a sidebar to main panel layout to guide the user within the book discovery process. To help with this process, I also employed a specific coding logic for when a user is successfully signed in, then the registration and login boxes are automatically hidden. I wanted to reduce any visual clutter so that users can get the full experience of the core purpose of the app, which is reviewing their personal reading history and exploring their own personalized recommendations.

Book Recommendation System

Login Here:

Create New Profile...

Your Name:

Sign Up

Rate a Book

Title:

Author:

Rating (1-5):

1

5

Welcome to the Book Engine!

Personalized Picks

Your History

Login Here:

Richa Gandhi

Create New Profile...

Richa Gandhi

Title:

Author:

Rating (1-5):

1

5

Submit

Delete My Profile

Books for Richa Gandhi

Personalized Picks

Balloons over Broadway
Melissa Sweet

First Little Readers Parent Pack
Deborah Schecter

How to Know a Person
David Brooks

Tom Lake
Ann Patchett

Verity
Colleen Hoover

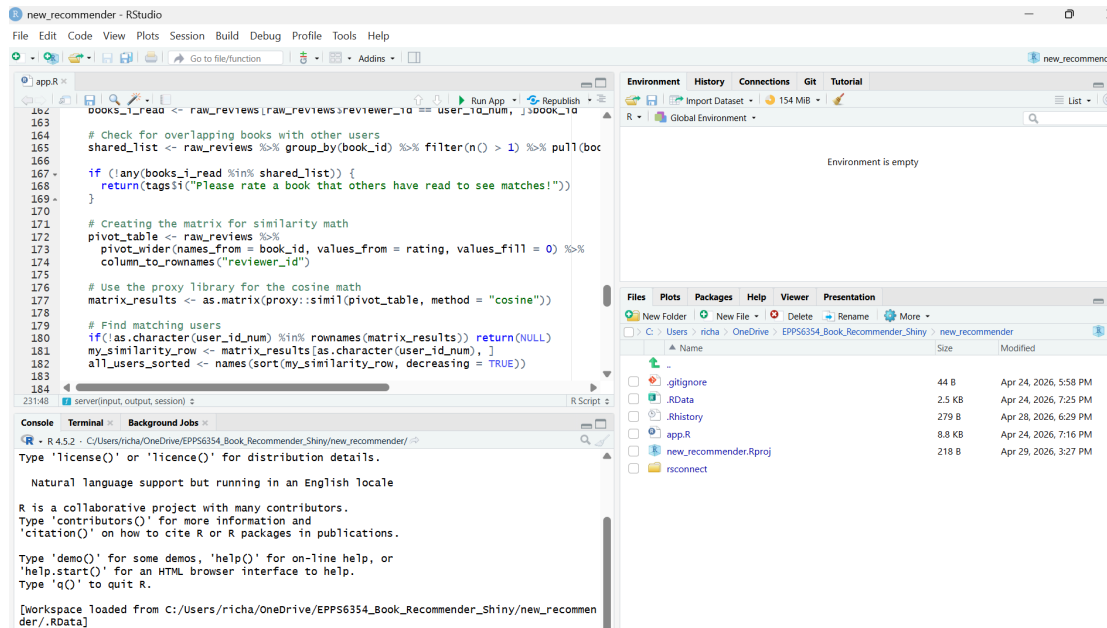
Your History

book_name	author_name	rating
And Then There Were None	Agatha Christie	5
The Hunger Games	Suzanne Collins	5
Anxious People	Fredrik Backman	5
Catching Fire	Suzanne Collins	4
A Little Life	Hanya Yanagihara	2

Study Expectations

The recommendation logic is built on collaborative filtering using the Cosine Similarity metric. In the approach for this metric, it identifies deep relationships between users and their historical rating activity, which consists of shared behavioral patterns and does not rely on metadata as much, such as publication dates or genres. The fundamental idea is derived from the fact that if two users agree on ratings/quality of books in the past, then the chances of them agreeing on others in the future are highly likely. Moving this from theory to a functional engine, all the relational data stored in the PostgreSQL reviews table is converted by the system into a matrix where every individual book is represented as a column, and every reader is represented as a row. Therefore, within this coordinate system, a user's entire taste profile is a vector signature. In this multidimensional space, every person is represented with an arrow pointing in a specific direction, and the calculation used between the vectors is the cosine of the angle given by this equation:

$$\text{Similarity (A, B) = Cosine} = \frac{A \cdot B}{\|A\| \|B\|}$$



The angle of these “arrows” is focused on more than the magnitude or raw length for the engine to achieve normalization. Different rating styles and individual user bias are accounted for from this distinction in the angle of the arrows. For example, a “generous” rater who frequently gives fives and a “harsh” rater who rarely gives a score above a three can still be mathematically similar if they like and dislike the same books relative to one another. Even if one vector is “longer” than the other from raw scores, the math can still recognize that vectors are pointing in the same direction. While two people can both give out a 5 rating to a single bestseller book, the cosine similarity takes into account the entire landscape of rating history from their shared library to ensure actual behavioral alignment. They must share a consistent pattern of liking and disliking the same sets of books for two users to be matched. In the equation shown above, there is a normalization component that stops high-volume raters from skewing the results and divides the shared ratings by the total magnitude of each user’s history. Instead of looking at just the raw volume of books

read it prioritizes a proportional pattern of taste to prevent a user who gives every book a 5 rating from accidentally matching the entire community immediately.

In the application code, a shared bridge logic is used, where this safeguard can identify when specific books have been rated by at least two different people in the database. It is required that when a new user joins the system to rate a book first in order for the similarity math to have a starting point for comparison and avoid any cold start problems. The database has hundreds of historical reviews along with pseudodata to increase density for the engine. The similarity scores range from 0 to 1 and are calculated for each user in the database. Individuals represent readers whose historical tastes closely align with the current profile of the user. A final scan of titles and neighbors rated highly is performed by the system, and filters out any books that the active user has encountered. It then ranks the title by its average neighbor score, where the final result displays the top 5 picks. Testing out the engine showed that extreme ratings of 1 star or 5 star can make a huge shift in the direction of the user's vector and will differ in what the system will show as most compatible book suggestions.

Once the similarity scores (ranging from 0 to 1) are calculated for every user in the database, the system identifies the Top 10 Nearest Neighbors for the active user. These individuals represent the readers whose historical tastes most closely align with the user's current profile. The engine then performs a final scan of the titles that these neighbors rated highly, filters out any books the active user has already encountered, and ranks the remaining titles by their average neighbor score. The final result is a display of the Top 5 Picks, ensuring that the recommendations are based on mathematically verified common ground rather than superficial or isolated data points. Testing revealed that this engine is highly sensitive to extreme ratings,

meaning that a single 1-star or 5-star review can significantly shift the direction of a user's vector and change who the system identifies as their most compatible reading match.

Book Recommendation System: https://rdgandhi22.shinyapps.io/new_recommender/

Future Scaling

The final app has been successfully hosted on Shiny. This transition proved that the R script can manage a connection to the cloud database. When looking at the future capabilities of expanding this project to be even more professional, I want the app to be able to serve multiple users because the system doesn't react well to multiple users on the site at once. Since this app serves as a conceptual idea, it is sufficient for now, but when I have plans to add more real/authentic book reviewers/ratings and take this on a larger scale this feature would be beneficial. Another feature I would like to incorporate is adding a content-based filter by scraping metadata like tags or genres, where the app could recommend books even if no one has rated that book yet. Finally, a feature that would make this system well-rounded in its capabilities would be detecting variance in a user's recent ratings, where the engine can identify declining interest and suggest a palate cleanser book to keep the experience fresh. A system that can pick up on recommending books in a different category, vibe matcher, and content-based filtering is the ultimate desired design by bookworms.

AI disclosure

AI-based tools, including ChatGPT and Google Gemini, were used solely for conceptual clarification during this database project, such as codes, and helped with the creation of the name "Vector Verse". They were not used in the writing of this paper or in the development of the project idea. All design decisions, schema development, and technical implementation were

completed based on my own understanding and effort. These tools served only as an aid and did not replace my original work or analysis.

References

OpenAI.(2026). “ChatGPT.” *OpenAI*. <https://chatgpt.com/>.

Preda, G. (2021). *Reddit book suggestions* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/gpreda/reddit-book-suggestions>

Tanwar, A. (2023). *Top 200 trending books with reviews* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/anshtanwar/top-200-trending-books-with-reviews>

Soumik. (2019). *Goodreads-books* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks>

dbdiagram.io. (n.d.). *dbdiagram.io* [Database design tool]. <https://dbdiagram.io>